Лабораторная работа № 1

Пайплайны LangChain и интерфейс к каталогу товаров
Выполнил: Михайлов Давид

1. Цель и устройство программы

Я реализовал поиск товаров по запросу на естественном языке. Модель qwen3.8-27b извлекает параметры, Pydantic проверяет их, а Python выполняет поиск в SQLite. Все результаты в отчёте взяты из сохранённых выводов ноутбука.

Цепочка: **запрос → промпт → LLM → ProductQuery → SQLite → товары**.

В ProductQuery три поля: категория (laptop, smartphone, tablet, other), максимальная цена (целое число ≥ 0 или null) и сортировка (price_asc / price_desc). По умолчанию ограничения цены нет, сортировка идёт по возрастанию. Неизвестная категория заменяется на other.

В SQL категория и цена подставляются через параметры ?, направление сортировки выбирается из разрешённого списка. Для other программа сообщает, что категория не поддерживается, и не ищет по всему каталогу. Вариант с инструментом использует ту же функцию поиска.

2. Базовая цепочка и структурированный ответ

Системное сообщение задаёт роль модели, пользовательское содержит query; переменная language задаёт язык ответа. Прямой вызов и цепочка со строковым парсером вернули:

Вызов	Ответ	Тип	Является строкой
LLM напрямую	Компьютеры и ноутбуки	AIMessage	Нет
LLM | StrOutputParser	Ноутбуки	TextAccessor	Да

TextAccessor наследует str. Названия категорий различаются, поскольку сделаны два отдельных вызова модели; парсер не переименовывает категорию. Batch из трёх запросов дал «Ноутбуки», «Смартфоны» и «Планшеты».

Для структурированного ответа я включил get_format_instructions() в системный промпт и добавил PydanticOutputParser. На запрос о смартфоне до 50 000 рублей, сначала дорогие, получен объект ProductQuery, сериализованный в JSON:

```
{"category":"smartphone","max_price":50000,"sort_by":"price_desc"}
```

В таблицах параметры записаны как **категория; максимальная цена; сортировка**. «Нет» означает null, «возр.» и «убыв.» соответствуют price_asc и price_desc.

Запрос	Параметры	Тест пройден
Найди ноутбуки дешевле 70 000 рублей	laptop; 70000; возр.	Да
Покажи самые дорогие смартфоны	smartphone; нет; убыв.	Да
Хочу посмотреть планшеты	tablet; нет; возр.	Да
Покажи ноутбуки до 50 тысяч	laptop; 50000; возр.	Да
Найди холодильники	other; нет; возр.	Да
Хочу недорогой смартфон	smartphone; нет; возр.	Да
Куплю планшет в районе тридцатки	tablet; нет; возр.	Нет

Прошли **6/7 тестов**. «Недорогой» не превратился в выдуманный бюджет, но для «тридцатки» модель вернула null вместо ожидаемых 30 000. При других запусках модель понимала, что это ограничение по цене.

3. Режим размышления: время, токены и ответ

Я сравнил два неоднозначных запроса с enable_thinking=False и True. Сначала — через LangChain:

Запрос	Thinking	Время, с	Токены	Парсер успешен
Куплю планшет в районе тридцатки	Выкл.	1.92	30	Да
Куплю планшет в районе тридцатки	Вкл.	2.73	209	Да
Хочу недорогой смартфон	Выкл.	0.39	31	Да
Хочу недорогой смартфон	Вкл.	2.31	105	Да

Все ответы разобраны. Для проверки блока размышления я выводил content_blocks. Там был только text. полные параметры в этой таблице не показаны. Для проверки рассуждения я повторил опыт через прямой клиент OpenAI:

Запрос	Thinking	Параметры	Время, с	Токены
Куплю планшет в районе тридцатки	Выкл.	tablet; нет; возр.	0.32	30
Куплю планшет в районе тридцатки	Вкл.	tablet; 30000; возр.	1.44	156
Хочу недорогой смартфон	Выкл.	smartphone; нет; возр.	0.37	31
Хочу недорогой смартфон	Вкл.	smartphone; нет; возр.	0.86	71

Через прямой API парсер также справился со всеми четырьмя ответами. При включённом thinking получен отдельный текст рассуждения; в content остался только итоговый JSON. Именно content передавался парсеру. Если склеить его с рассуждением, свободный текст может нарушить разбор по схеме.

ChatOpenAI в данном подключении не сохранил нестандартное поле рассуждения. Отсутствие этого поля в AIMessage не означает, что модель не размышляла. Его нет, так как интеграция ChatOpenAI не сохраняет нестандартные поля сторонних серверов, включая reasoning_content, reasoning и reasoning_details. Прямой клиент извлекал reasoning_content, либо reasoning.

Включение thinking увеличило время и число токенов. Для смартфона параметры не изменились; для планшета цена изменилась с null на 30 000. Это результат отдельных вызовов, а не статистика качества.

4. Поиск в SQLite

В каталоге 15 товаров: по пять ноутбуков, смартфонов и планшетов. RunnableLambda включает функцию поиска в LCEL. Проверка сравнивает параметры и ID найденных товаров с ожидаемыми, включая порядок сортировки.

Запрос	Параметры	Найденные товары и цены, руб.	Верно
Ноутбуки дешевле 70000 рублей	laptop; 70000; возр.	Lenovo IdeaPad 3: 45000, ASUS Vivobook 15: 55000, Acer Aspire 5: 65000	Да
Ноутбуки до 50000 рублей	laptop; 50000; возр.	Lenovo IdeaPad 3: 45000	Да
Ноутбуки до 40000 рублей	laptop; 40000; возр.	Пустой список	Да
Все ноутбуки, сначала дорогие	laptop; нет; убыв.	Apple MacBook Air: 120000, HP Pavilion 15: 85000, Acer Aspire 5: 65000, ASUS Vivobook 15: 55000, Lenovo IdeaPad 3: 45000	Да
Смартфоны до 30000 рублей	smartphone; 30000; возр.	Xiaomi Redmi Note 13: 18000, Samsung Galaxy A35: 28000	Да
Все смартфоны, сначала дорогие	smartphone; нет; убыв.	Apple iPhone 15: 90000, Samsung Galaxy S24: 75000, Google Pixel 8a: 45000, Samsung Galaxy A35: 28000, Xiaomi Redmi Note 13: 18000	Да
Смартфоны до 10000 рублей	smartphone; 10000; возр.	Пустой список	Да
Планшеты до 30 тысяч, сначала дешёвые	tablet; 30000; возр.	Lenovo Tab M11: 15000, Samsung Galaxy Tab A9: 22000, Xiaomi Pad 6: 29000	Да
Все планшеты, сначала дорогие	tablet; нет; убыв.	Samsung Galaxy Tab S9: 70000, Apple iPad: 45000, Xiaomi Pad 6: 29000, Samsung Galaxy Tab A9: 22000, Lenovo Tab M11: 15000	Да
Найди холодильники	other; нет; возр.	Категория не поддерживается	Да

Результат — **10/10 проверок**. Поиск корректно вернул несколько товаров, один товар или пустой список. Для холодильников выдано сообщение о неподдерживаемой категории.

5. Ответ без базы и вызов инструмента

5.1. Три запроса без доступа к каталогу

Я передал модели те же запросы без инструмента и записей SQLite. Во всех трёх ответах она признала отсутствие доступа к каталогу.

Запрос	SQLite, руб.	Ответ без базы
Ноутбуки дешевле 70000 рублей	Lenovo IdeaPad 3: 45000, ASUS Vivobook 15: 55000, Acer Aspire 5: 65000	Не назвала товары каталога. Предложила общие характеристики ноутбуков и способы поиска.
Смартфоны до 30000 рублей	Xiaomi Redmi Note 13: 18000, Samsung Galaxy A35: 28000	Привела ориентиры Redmi Note 13 (18–22 тыс.), Galaxy A35 5G (25–29 990) и ещё четыре модели, которых нет в выдаче. Точные цены и полнота каталога не подтверждены.
Планшеты до 30 тысяч, сначала дешёвые	Lenovo Tab M11: 15000, Samsung Galaxy Tab A9: 22000, Xiaomi Pad 6: 29000	Попросила предоставить список товаров. Упомянутые Tab A8, Xiaomi Pad 5 и Honor Pad X8 были гипотетическим примером, а не результатом поиска.

Общие знания модели не заменяют каталог: даже совпадение семейства смартфонов не подтверждает наличие конкретного товара. Цены в её рекомендациях — часть ответа модели, а не проверенные рыночные данные.

5.2. Поиск как инструмент

Инструмент search_products связан с ProductQuery через args_schema и передан модели методом bind_tools. Программа сначала проверяет аргументы, затем вызывает поиск. Его результат обратно модели не отправляется.

Запрос	Вызов после проверки	Найдено
Найди ноутбуки дешевле 70 000 рублей	laptop; 70000; возр.	3
Покажи самые дорогие смартфоны	max_price="None": ошибка типа	Поиска нет
Хочу посмотреть планшеты	tablet; нет; возр.	5
Покажи ноутбуки до 50 тысяч	laptop; 50000; возр.	1
Найди холодильники	tool_calls=[]; отказ	Поиска нет
Хочу недорогой смартфон	smartphone; 20000; возр.	1
Куплю планшет в районе тридцатки	tablet; 30000; возр.	3

На холодильниках модель не вызвала инструмент — программа не запускала SQL. На смартфонах «сначала дорогие» модель передала строку "None" вместо JSON null. Проверка отклонила аргументы, SQL не запустился. В успешных вызовах пропущенные поля заполнились значениями по умолчанию.

Но «недорогому смартфону» инструмент назначил бюджет 20 000 без указания пользователя. Это смысловая ошибка, хотя схема её пропустила: число допустимо, но взято из предположения модели.

Этап	JSON в тексте ответа	Tool calling
Схема	ProductQuery и инструкции в system	ProductQuery в args_schema
Выбор значений	LLM генерирует JSON	LLM формирует аргументы
Разбор	PydanticOutputParser	API / LangChain формирует tool_calls
Проверка	Pydantic в парсере	ProductQuery.model_validate
Действие	Python-функция после проверки	Программа вызывает инструмент

6. Ошибки, повторные попытки и журнал

Для проверки я подготовил четыре невалидных ответа. Во всех случаях парсер выдал OutputParserException; ниже указана исходная причина из __cause__.

Ответ / поле	Причина	Где обнаружена
{"category":"laptop",,"max_price":50000}	JSONDecodeError	Разбор JSON: лишняя запятая
sort_by="popular"	ValidationError: literal_error	Проверка перечисления
max_price="дёшево"	ValidationError: int_parsing	Проверка типа цены
max_price=-100	ValidationError: greater_than_equal	Проверка цены ≥ 0

До успешной валидации поиск не выполняется. with_retry оборачивает только генерацию и парсинг, разрешая максимум три попытки. После их исчерпания программа сообщает: «Не удалось разобрать ответ модели. Уточните запрос.»

Чтобы результат не зависел от удачного ответа реальной LLM, fake_llm возвращает заранее заданные сообщения: сначала плохой JSON, затем правильный. В другом сценарии плохой ответ повторяется. Реальная модель в этих тестах не вызывается.

Сценарий	Попытки	Вызовы поиска	Статус
Ошибка → успех, без retry	1	0	error
Ошибка → успех, с retry	2	1	ok
Всегда ошибка, без retry	1	0	error
Всегда ошибка, с retry	3	0	error

Журнал сценария с ошибкой и успехом на второй попытке:

```
1  llm_start
1  parse_error      Некорректный JSON
2  llm_start
2  parse_ok
2  validation_ok    laptop; 70000; price_asc
2  search_start
2  search_done      Найдено 3 товара
```

SQL запущен один раз после validation_ok. При трёх ошибках в журнале нет запуска поиска. Повтор использует тот же промпт, без передачи модели предыдущей ошибки.

На реальных запросах журнал также показал: смартфон до 25 000 — один результат; все смартфоны после подтверждения — пять; «машины» — unsupported_category; запрос без цены после отказа пользователя — cancelled. Последний статус означает отмену пользователем, а не ошибку модели.

7. Уточнения: параметры или история

Я сравнил передачу последнего валидного ProductQuery и всей истории диалога. Неуказанные поля сохраняются, смена категории не сбрасывает цену и сортировку, а просьба убрать ограничение устанавливает null.

Диалог шаг	Реплика	Параметры обоих вариантов	Токены: состояние / история	Товаров
1.1	Покажи ноутбуки до 80 тысяч	laptop; 80000; возр.	430 / 422	3
1.2	А сначала дорогие?	laptop; 80000; убыв.	444 / 475	3
1.3	А что есть среди смартфонов?	smartphone; 80000; убыв.	447 / 531	4
2.1	Покажи планшеты до 30 тысяч	tablet; 30000; возр.	431 / 423	3
2.2	Убери ограничение цены	tablet; нет; возр.	444 / 476	5
2.3	Теперь ноутбуки, сначала дорогие	laptop; нет; убыв.	443 / 527	5

На всех шести шагах оба подхода дали ожидаемые параметры: **6/6 у каждого**. Всего передано 2639 входных токенов при хранении параметров и 2854 при передаче истории. Экономия — **215 токенов, или 7,5%**. К третьему шагу история растёт до 531 и 527 токенов, а состояние занимает 447 и 443. Для этих коротких диалогов явное состояние оказалось экономнее без потери точности.

8. Вывод

В ходе работы я освежил навыки с LangChain. Pydantic отсеивает сломанный JSON, неверные типы, перечисления и отрицательные цены, но не проверяет смысл извлечения. Пример — выдуманный бюджет для «недорогого» смартфона.

Схема тоже упрощает запросы: «дешевле» и «до» превращаются в <=, а «в районе» нельзя точно выразить одной верхней границей. Ещё одна реальная ошибка — строка "None" в аргументах инструмента вместо null. В отличие от выдуманного бюджета, её Pydantic обнаружил и остановил поиск.

Основной результат — модель интерпретирует текст, а каталог остаётся источником товаров и цен. Поиск прошёл 10/10 проверок, structured output — 6/7 (при повторных запусках 7/7). Retry восстановил выполнение после плохого ответа и не запускал SQL при исчерпании попыток. Уточнения сохранили нужные параметры в обоих вариантах.